AI ASSISTED CODING ASSIGNMENT NO-14.1

Name:SIRIVELLA SANJANA

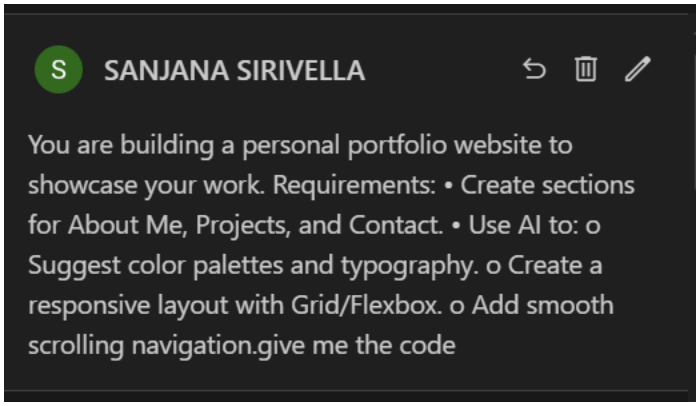
ROLLNO: 2403A510D4

BATCH NO:05

Task1– Portfolio Website Design

You are building a personal portfolio website to showcase your work.
Requirements:
• Create sections for About Me, Projects, and Contact.
• Use AI to:
- Suggest color palettes and typography.
- Create a responsive layout with Grid/Flexbox.
- Add smooth scrolling navigation

PROMPT:



CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" c
    <title>Sirivella Sanjan
    <link rel="preconnect"
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600;700&display=
    <style>
        /* AI-Suggested Color Palette & Base Styles */
        :root {
            --bg-color: #f8f9fa;
            --text-color: #495057;
            --header-color: #343a40;
            --primary-color: #007bff;
            --card-bg: #ffffff;
            --shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }

        html {
            scroll-behavior: smooth; /* Enables smooth scrolling */
        }

        body {
            font-family: 'Poppins', sans-serif;
            margin: 0;
            background-color: var(--bg-color);
            color: var(--text-color);
            line-height: 1.6;
```

This attribute specifies the URL of the linked resource. A URL can be absolute or relative.

Widely available across major browsers (Baseline since 2015)

```
        }

        .container {
            max-width: 1100px;
            margin: 0 auto;
            padding: 0 20px;
        }

        /* Header and Navigation */
        header {
            background: var(--card-bg);
            box-shadow: var(--shadow);
            padding: 1rem 0;
            position: sticky;
            top: 0;
            z-index: 100;
        }

        nav {
            display: flex;
            justify-content: space-between;
            align-items: center;
        }

        nav .logo {
            font-size: 1.5rem;
```

```
54
55      nav .logo {
56          font-size: 1.5rem;
57          font-weight: 700;
58          color: var(--header-color);
59      }
60
61      nav ul {
62          list-style: none;
63          display: flex;
64          gap: 2rem;
65          margin: 0;
66      }
67
68      nav a {
69          text-decoration: none;
70          color: var(--header-color);
71          font-weight: 600;
72          transition: color 0.3s ease;
73      }
74
75      nav a:hover {
76          color: var(--primary-color);
77      }
78
79      /* Sections */
```

```
79      /* Sections */
80      section {
81          padding: 60px 0;
82      }
83
84      h1, h2 {
85          color: var(--header-color);
86          text-align: center;
87          margin-bottom: 40px;
88      }
89
90      /* About Me Section */
91      #about p {
92          text-align: center;
93          max-width: 700px;
94          margin: 0 auto;
95          font-size: 1.1rem;
96      }
97
98      /* Projects Section - Responsive Grid */
99      .project-grid {
100         display: grid;
101         grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
102         gap: 2rem;
103     }
```

```
        .project-card {
            background: var(--card-bg);
            border-radius: 8px;
            box-shadow: var(--shadow);
            overflow: hidden;
            transition: transform 0.3s ease;
        }

        .project-card:hover {
            transform: translateY(-5px);
        }

        .project-card img {
            width: 100%;
            height: 200px;
            object-fit: cover;
        }

        .project-card-content {
            padding: 1.5rem;
        }

        .project-card h3 {
            margin-top: 0;
            color: var(--header-color);
        }
```

```
        /* Contact Section */
        #contact {
            text-align: center;
        }
    </style>
</head>
<body>
    <header class="container">
        <nav>
            <div class="logo">Sirivella Sanjana</div>
            <ul>
                <li><a href="#about">About Me</a></li>
                <li><a href="#projects">Projects</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
        </nav>
    </header>

    <main class="container">
        <section id="about">
            <h1>About Me</h1>
            <p>Hi! I'm Sirivella Sanjana, a passionate web developer with a love for creating
        </section>

        <section id="projects">
            <h2>My Projects</h2>
            <div class="project-grid">
```

Ln 3, Col 7 Spaces: 4 UTF-8 CRLF {} HTML Port : 5501

```
            <div class="project-grid">
                <!-- Project 1 -->
                <div class="project-card">
                    <img src="https://via.placeholder.com/400x250/007bff/ffffff?text=Project+O
                    <div class="project-card-content">
                        <h3>Project One</h3>
                        <p>A brief description of the project, its purpose, and the technologi
                    </div>
                </div>
                <!-- Project 2 -->
                <div class="project-card">
                    <img src="https://via.placeholder.com/400x250/343a40/ffffff?text=Project+T
                    <div class="project-card-content">
                        <h3>Project Two</h3>
                        <p>A brief description of the project. This could be a data visualizat
                    </div>
                </div>
                <!-- Project 3 -->
                <div class="project-card">
                    <img src="https://via.placeholder.com/400x250/6c757d/ffffff?text=Project+T
                    <div class="project-card-content">
                        <h3>Project Three</h3>
                        <p>A brief description of the project. Perhaps a personal blog platfor
                    </div>
                </div>
            </div>
```

```
            </div>
        </section>

        <section id="contact">
            <h2>Get In Touch</h2>
            <p>I'm currently open to new opportunities. Feel free to reach out!</p>
            <p><strong>Email:</strong> <a href="mailto:sanjana@example.com">sanjana@example.co
        </section>
    </main>

    <footer>
        <p style="text-align: center; padding: 20px 0; color: var(--text-color);">&copy; 2024
    </footer>
</body>
</html>
```

OBSERVATIONS:

Strengths of the Code

- **Modern CSS:** You've made great use of CSS variables (`:root`) for the color palette, which makes theming and future changes incredibly easy. The use of `display: grid` with `repeat(auto-fit, minmax(300px, 1fr))` for the project section is a fantastic, modern way to create a responsive grid without needing explicit media queries.
- **Semantic HTML:** The structure correctly uses semantic elements like `<header>`, `<nav>`, `<main>`, `<section>`, and `<footer>`. This is great for both SEO and accessibility, as it gives clear meaning to each part of your page.
- **Performance:** Using `scroll-behavior: smooth;` in CSS is a highly efficient, performant way to achieve smooth scrolling without any JavaScript, which is a smart choice.
- **Readability:** The code is well-commented, and the class names are clear and descriptive (e.g., `.project-grid`, `.project-card`).

Suggestions for Improvement

While the code is already very good, here are a few suggestions to elevate it even further, focusing on robustness and accessibility.

1. Add a Mobile-Friendly Navigation (Hamburger Menu)

On smaller screens, the navigation links might wrap or overflow. A common solution is to collapse them into a "hamburger" menu. This requires a bit of HTML, CSS, and a few lines of JavaScript.

Prompts to try

1. Add a Mobile-Friendly Navigation (Hamburger Menu)

On smaller screens, the navigation links might wrap or overflow. A common solution is to collapse them into a "hamburger" menu. This requires a bit of HTML, CSS, and a few lines of JavaScript.

Suggestion: I can add the necessary HTML for a menu button, update the CSS with a media query to hide the links and show the button on mobile, and write a small JavaScript snippet to toggle the menu's visibility when the button is clicked.

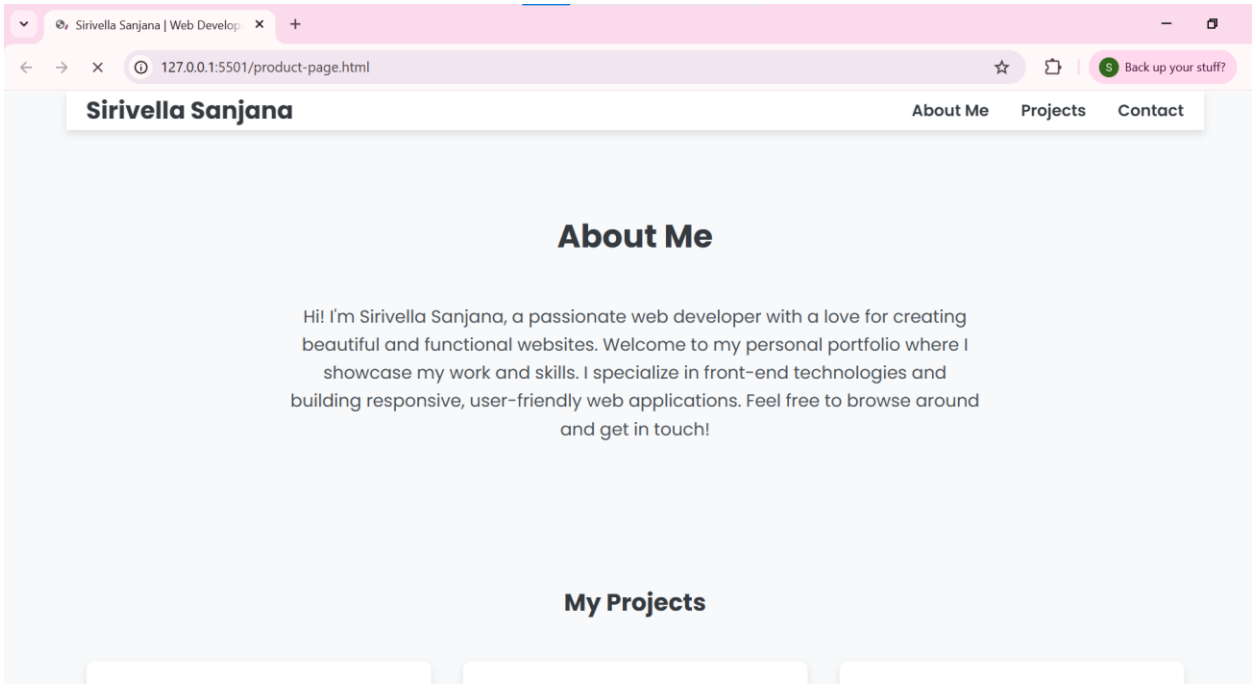
2. Improve Accessibility and User Experience

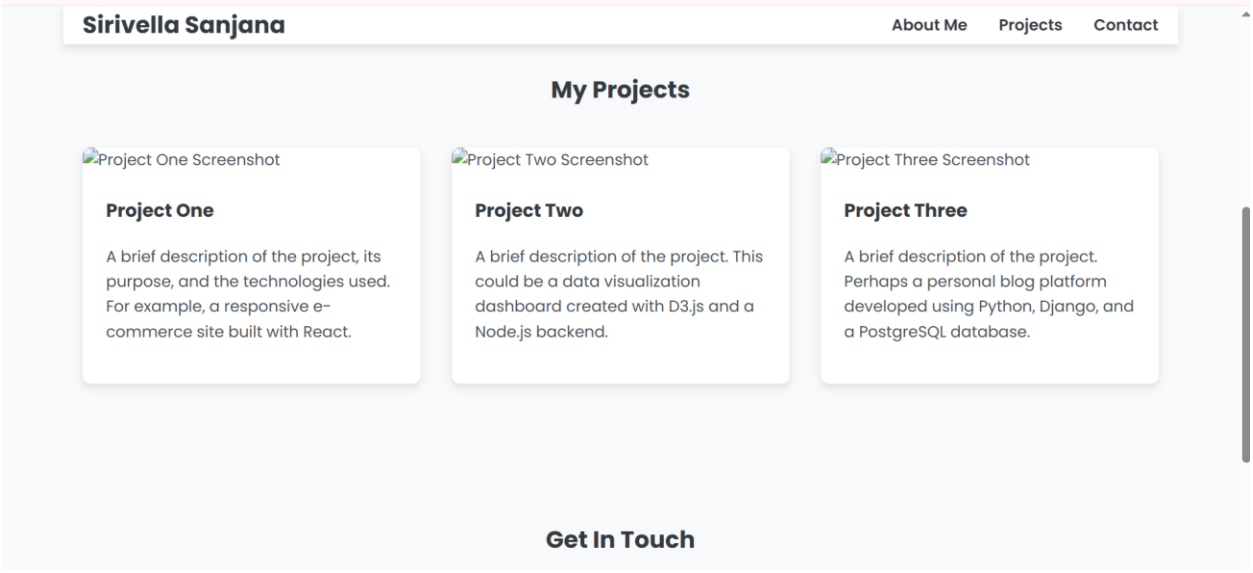
A few small tweaks can make the site more accessible and user-friendly.

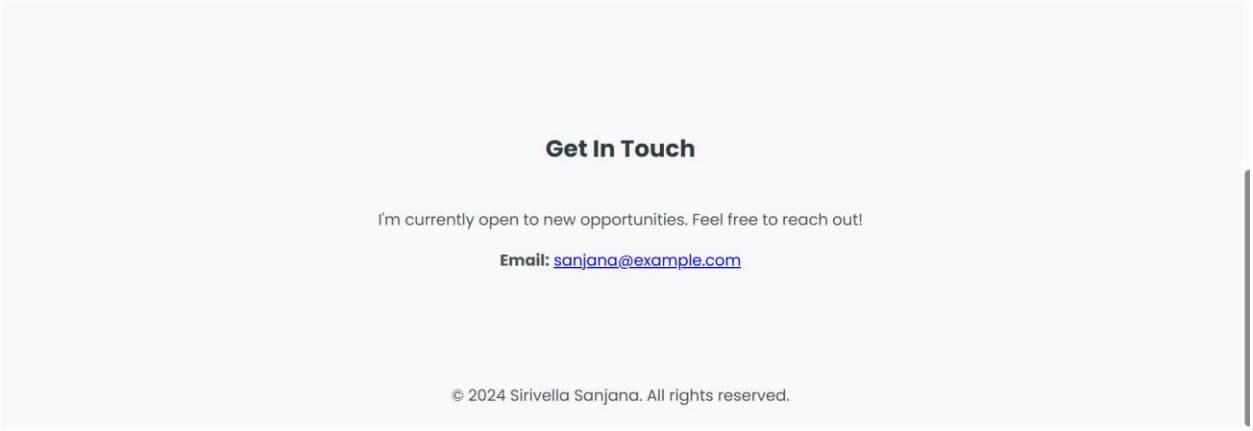
- **Link Contrast:** The primary blue color (`#007bff`) on the light background has a contrast ratio that is slightly below the recommended standard for normal-sized text. We can darken it slightly to meet accessibility guidelines.
- **Interactive Logo:** The logo in the navigation bar is currently static text. Making it a link that takes the user back to the top of thepage is a standard and helpful convention.

I've prepared a `diff` that implements these accessibility improvements and also adds a more robust CSS reset.

OUTPUT:



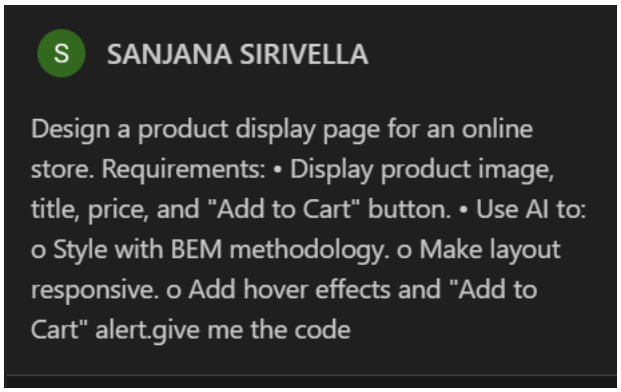

Sirivella Sanjana
About Me
Projects
Contact
My Projects
Project One Screenshot
Project One
A brief description of the project, its purpose, and the technologies used. For example, a responsive e-commerce site built with React.
Project Two Screenshot
Project Two
A brief description of the project. This could be a data visualization dashboard created with D3.js and a Node.js backend.
Project Three Screenshot
Project Three
A brief description of the project. Perhaps a personal blog platform developed using Python, Django, and a PostgreSQL database.
Get In Touch


Get In Touch
I'm currently open to new opportunities. Feel free to reach out!
Email: sanjana@example.com
© 2024 Sirivella Sanjana. All rights reserved.

TASK-02: Online Store Product Page

Design a product display page for an online store.
Requirements:
• Display product image, title, price, and "Add to Cart" button.
• Use AI to:
- Style with BEM methodology.
- Make layout responsive.
- Add hover effects and "Add to Cart" alert.

PROMPT:



CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Display Page</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Lato:wght@400;700&display=swap" rel="styl
    <style>
        /* AI-Suggested Design & Base Styles */
        :root {
            --bg-color: #f4f4f4;
            --card-color: #ffffff;
            --text-color: #333333;
            --primary-color: #17a2b8;
            --primary-hover-color: #138496;
            --shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
            --shadow-hover: 0 6px 20px rgba(0, 0, 0, 0.15);
        }

        body {
            font-family: 'Lato', sans-serif;
            background-color: var(--bg-color);
            color: var(--text-color);
            margin: 0;
            /* Use Flexbox to center the product card */
            display: flex;
            justify-content: center;
            align-items: center;
```

```
<head>
    <style>
        body {
            min-height: 100vh;
            padding: 20px;
            box-sizing: border-box;
        }

        /* BEM Block: .product-card */
        .product-card {
            background-color: var(--card-color);
            border-radius: 12px;
            box-shadow: var(--shadow);
            overflow: hidden;
            max-width: 350px;
            width: 100%;
            text-align: center;
            transition: transform 0.3s ease, box-shadow 0.3s ease;
        }

        /* AI-Suggested Hover Effect */
        .product-card:hover {
            transform: translateY(-8px);
            box-shadow: var(--shadow-hover);
        }

        /* BEM Element: .product-card__image */
        .product-card__image {
            width: 100%;
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} HTML Port : 5501

```css
        /* BEM Element: .product-card__image */
        .product-card__image {
            width: 100%;
            height: auto;
            display: block;
        }

        /* BEM Element: .product-card__info */
        .product-card__info {
            padding: 25px;
        }

        /* BEM Element: .product-card__title */
        .product-card__title {
            font-size: 1.5rem;
            font-weight: 700;
            margin: 0 0 10px 0;
        }

        /* BEM Element: .product-card__price */
        .product-card__price {
            font-size: 1.75rem;
            font-weight: 700;
            color: var(--primary-color);
            margin: 0 0 20px 0;
        }
```

```css
        /* BEM Element: .product-card__button */
        .product-card__button {
            display: inline-block;
            width: 100%;
            padding: 12px 20px;
            border: none;
            border-radius: 8px;
            font-size: 1rem;
            font-weight: 700;
            color: #fff;
            cursor: pointer;
            transition: background-color 0.3s ease;
            box-sizing: border-box;
        }

        /* BEM Modifier: .product-card__button--primary */
        .product-card__button--primary {
            background-color: var(--primary-color);
        }

        .product-card__button--primary:hover {
            background-color: var(--primary-hover-color);
        }

    </style>
</head>
```

```
<body>

    <div class="product-card">
        <img src="https://placehold.co/600x400/17a2b8/ffffff?text=Product" alt="A sample product i
        <div class="product-card__info">
            <h1 class="product-card__title">Modern Widget</h1>
            <p class="product-card__price">$49.99</p>
            <button id="addToCartBtn" class="product-card__button product-card__button--primary">
                Add to Cart
            </button>
        </div>
    </div>

    <script>
        // AI-Suggested "Add to Cart" Alert
        document.getElementById('addToCartBtn').addEventListener('click', function() {
            // Get product details from the card
            const title = document.querySelector('.product-card__title').textContent;
            const price = document.querySelector('.product-card__price').textContent;

            // Show a confirmation alert
            alert(`"${title}" at ${price} has been added to your cart!`);
        });
    </script>

</body>
</html>
```

Observations:

Strengths of the Code

- **Excellent CSS Architecture:** You've correctly implemented the BEM (Block, Element, Modifier) methodology. This makes the CSS highly modular, reusable, and easy to maintain, especially as the project grows. The class names like `.product-card__title` are perfectly clear.
- **Modern and Maintainable Styling:** The use of CSS Custom Properties (variables) in the `:root` is a best practice. It centralizes your design tokens (colors, shadows), making it incredibly easy to change the entire look and feel of the component from one place.
- **Effective Responsive Design:** The combination of `max-width: 350px` and `width: 100%` on the `.product-card` is a simple yet powerful technique for making the component fluid on small screens and fixed on larger ones. Using Flexbox on the `body` to center the card is also a great modern approach.
- **Good User Experience (UX):** The hover effects on both the card (`transform: translateY(-8px)`) and the button provide satisfying visual feedback to the user, making the page feel more interactive and alive.
- **Clean HTML Structure:** The HTML is semantic and well-organized. Using an `<h1>` for the title and a `<p>` for the price is logical. The `alt` attribute on the image is also present, which is good for accessibility.
- **Efficient JavaScript:** The script is minimal and effective. It correctly targets the button and uses an `alert()` to provide immediate feedback, which fulfills the requirement perfectly.

Suggestions for Improvement

The code is already very strong, but here are a couple of minor suggestions that could enhance it even further, focusing on accessibility and user experience.

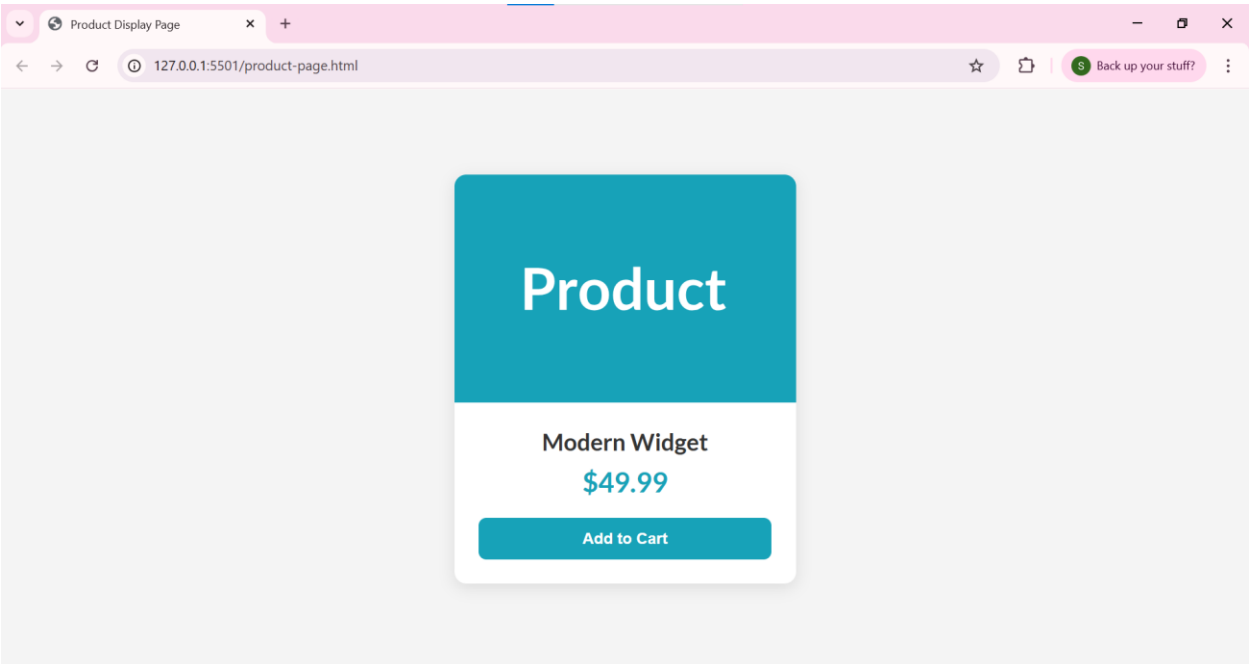
1. Improve Accessibility with Semantic HTML

The product title is currently an `<h1>`. In a real-world store, a page might have multiple products, and there should only be one `<h1>` per page (usually the page's main title, like "Our Products"). It's better to use a different heading level like `<h2>` for individual product titles.

2. Enhance the "Add to Cart" Notification

A browser `alert()` is functional, but it can be disruptive to the user experience. A more modern approach is to use a non-blocking "toast" notification that appears on the screen for a few seconds and then fades away. This would require a little more HTML and CSS.

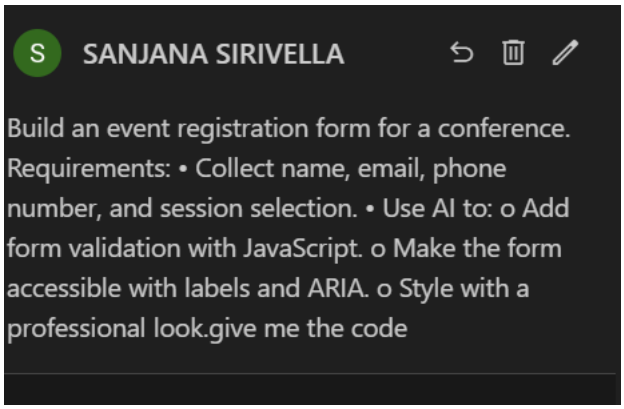
Output:



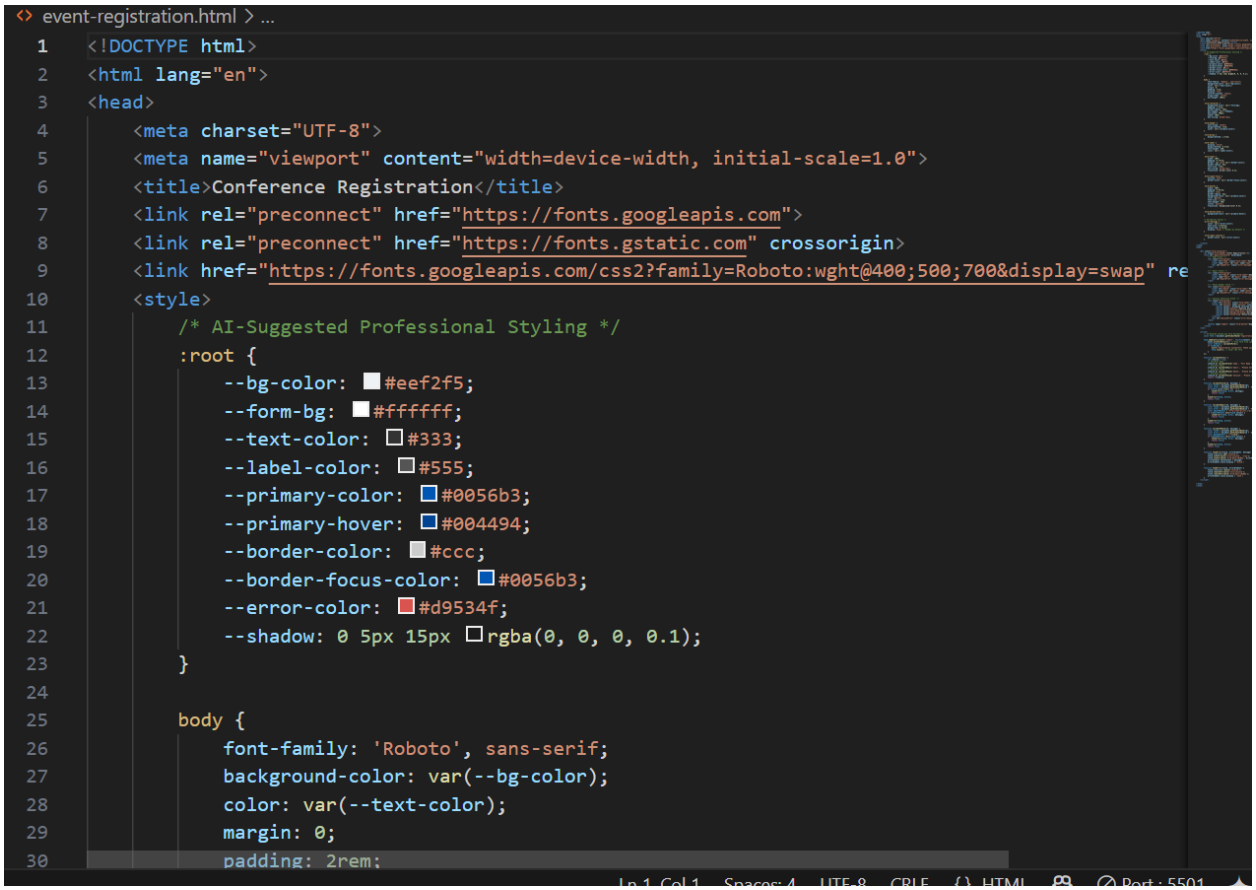
TASK-3: Event Registration Form

Build an event registration form for a conference.
Requirements:
• Collect name, email, phone number, and session selection.
• Use AI to:
- Add form validation with JavaScript.
- Make the form accessible with labels and ARIA.
- Style with a professional look

PROMPT:



CODE:



```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Conference Registration</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;500;700&display=swap" re
    <style>
        /* AI-Suggested Professional Styling */
        :root {
            --bg-color: #eef2f5;
            --form-bg: #ffffff;
            --text-color: #333;
            --label-color: #555;
            --primary-color: #0056b3;
            --primary-hover: #004494;
            --border-color: #ccc;
            --border-focus-color: #0056b3;
            --error-color: #d9534f;
            --shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
        }

        body {
            font-family: 'Roboto', sans-serif;
            background-color: var(--bg-color);
            color: var(--text-color);
            margin: 0;
            padding: 2rem;
```

```
10      <style>
24
25          body {
26              font-family: 'Roboto', sans-serif;
27              background-color: var(--bg-color);
28              color: var(--text-color);
29              margin: 0;
30              padding: 2rem;
31              display: flex;
32              justify-content: center;
33              align-items: center;
34              min-height: 100vh;
35          }
36
37          .form-container {
38              background-color: var(--form-bg);
39              padding: 2.5rem;
40              border-radius: 10px;
41              box-shadow: var(--shadow);
42              max-width: 500px;
43              width: 100%;
44              box-sizing: border-box;
45          }
46
47          .form-header {
48              text-align: center;
49              margin-bottom: 2rem;
50              color: var(--primary-color);
```

```
46
47          .form-header {
48              text-align: center;
49              margin-bottom: 2rem;
50              color: var(--primary-color);
51          }
52
53          .form-group {
54              margin-bottom: 1.5rem;
55          }
56
57          .form-label {
58              display: block;
59              margin-bottom: 0.5rem;
60              font-weight: 500;
61              color: var(--label-color);
62          }
63
64          .form-input {
65              width: 100%;
66              padding: 0.75rem;
67              border: 1px solid var(--border-color);
68              border-radius: 5px;
69              font-size: 1rem;
70              box-sizing: border-box;
71              transition: border-color 0.3s;
72          }
```

```
        .form-input {
            width: 100%;
            padding: 0.75rem;
            border: 1px solid var(--border-color);
            border-radius: 5px;
            font-size: 1rem;
            box-sizing: border-box;
            transition: border-color 0.3s;
        }

        .form-input:focus {
            outline: none;
            border-color: var(--border-focus-color);
        }

        .form-button {
            width: 100%;
            padding: 0.85rem;
            border: none;
            border-radius: 5px;
            background-color: var(--primary-color);
            color: white;
            font-size: 1.1rem;
            font-weight: 700;
            cursor: pointer;
            transition: background-color 0.3s;
```

```
        .form-button:hover {
            background-color: var(--primary-hover);
        }

        /* Validation Styles */
        .error-message {
            color: var(--error-color);
            font-size: 0.875rem;
            margin-top: 0.25rem;
            display: none; /* Hidden by default */
        }

        .form-input.invalid {
            border-color: var(--error-color);
        }

    </style>
</head>
<body>

    <div class="form-container">
        <h1 class="form-header">Event Registration</h1>
        <form id="registrationForm" novalidate>
            <!-- Name Field -->
            <div class="form-group">
                <label for="name" class="form-label">Full Name</label>
```

```
12      <div class="form-container">
13          <h1 class="form-header">Event Registration</h1>
14          <form id="registrationForm" novalidate>
15              <!-- Name Field -->
16              <div class="form-group">
17                  <label for="name" class="form-label">Full Name</label>
18                  <input type="text" id="name" name="name" class="form-input" placeholder="e.g., Sa
19                  <div id="nameError" class="error-message" aria-live="polite"></div>
20              </div>
21
22              <!-- Email Field -->
23              <div class="form-group">
24                  <label for="email" class="form-label">Email Address</label>
25                  <input type="email" id="email" name="email" class="form-input" placeholder="you@e
26                  <div id="emailError" class="error-message" aria-live="polite"></div>
27              </div>
28
29              <!-- Phone Number Field -->
30              <div class="form-group">
31                  <label for="phone" class="form-label">Phone Number</label>
32                  <input type="tel" id="phone" name="phone" class="form-input" placeholder="e.g., 9
33                  <div id="phoneError" class="error-message" aria-live="polite"></div>
34              </div>
35
36              <!-- Session Selection Field -->
37              <div class="form-group">
38                  <label for="session" class="form-label">Choose a Session</label>
39                  <select id="session" name="session" class="form-input" required aria-required="tr
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} HTML Port : 5501

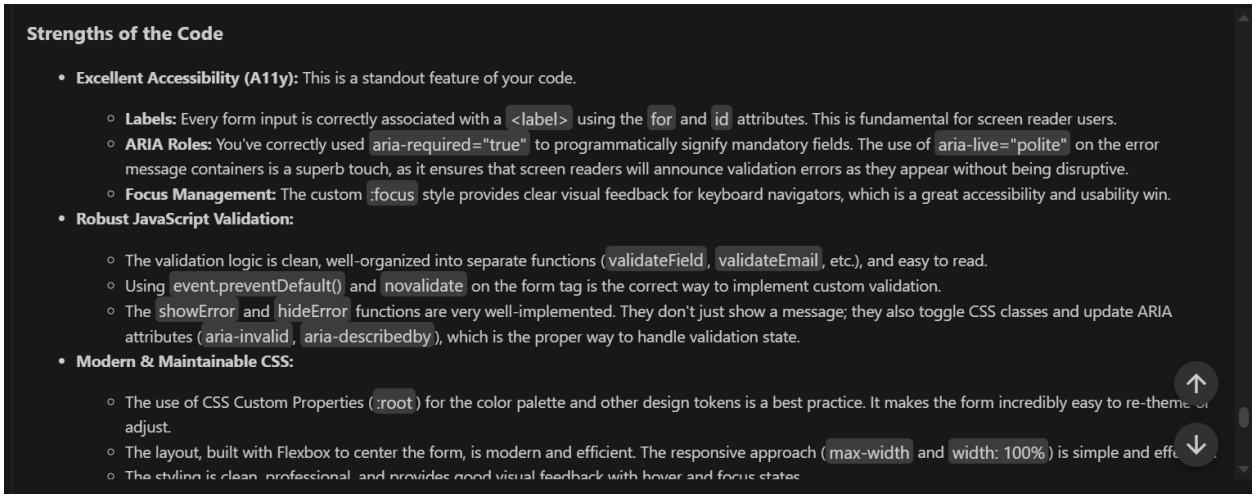
```
110 :body>
112     <div class="form-container">
114         <form id="registrationForm" novalidate>
135
136             <!-- Session Selection Field -->
137             <div class="form-group">
138                 <label for="session" class="form-label">Choose a Session</label>
139                 <select id="session" name="session" class="form-input" required aria-required="tru
140                     <option value="" disabled selected>Select a session...</option>
141                     <option value="keynote">Opening Keynote</option>
142                     <option value="tech-deep-dive">Technical Deep Dive</option>
143                     <option value="design-workshop">Design Thinking Workshop</option>
144                     <option value="networking">Networking Mixer</option>
145                 </select>
146                 <div id="sessionError" class="error-message" aria-live="polite"></div>
147             </div>
148
149             <button type="submit" class="form-button">Register Now</button>
150         </form>
151     </div>
152
153     <script>
154         // AI-Powered JavaScript Form Validation
155         const form = document.getElementById('registrationForm');
156
157         form.addEventListener('submit', function(event) {
158             event.preventDefault(); // Stop form from submitting
159             const isValid = validateForm();
160             if (isValid) {
```

```
        if (isValid) {
            alert('Registration successful! Thank you.');
            form.reset(); // Clear the form
        }
    });

    function validateForm() {
        let isValid = true;
        // Validate Name
        isValid &= validateField('name', 'Full Name is required.');
        // Validate Email
        isValid &= validateEmail('email', 'Please enter a valid email address.');
        // Validate Phone
        isValid &= validatePhone('phone', 'Please enter a valid 10-digit phone number.');
        // Validate Session
        isValid &= validateField('session', 'Please select a session.');
        return !!isValid;
    }

    function validateField(id, message) {
        const field = document.getElementById(id);
        const error = document.getElementById(id + 'Error');
        if (field.value.trim() === '') {
            showError(field, error, message);
            return false;
```

```
    function validateField(id, message) {
            return false;
        }
        hideError(field, error);
        return true;
    }

    function validateEmail(id, message) {
        const field = document.getElementById(id);
        const error = document.getElementById(id + 'Error');
        const emailPattern = /^[^@\s]+@[^@\s]+\.[^@\s]+$/;
        if (!emailPattern.test(field.value)) {
            showError(field, error, message);
            return false;
        }
        hideError(field, error);
        return true;
    }

    function validatePhone(id, message) {
        const field = document.getElementById(id);
        const error = document.getElementById(id + 'Error');
        const phonePattern = /^\d{10}$/;
        if (!phonePattern.test(field.value)) {
            showError(field, error, message);
            return false;
```

```
        function validatePhone(id, message) {
            }
            hideError(field, error);
            return true;
        }

        function showError(field, errorElement, message) {
            field.classList.add('invalid');
            field.setAttribute('aria-invalid', 'true');
            field.setAttribute('aria-describedby', errorElement.id);
            errorElement.textContent = message;
            errorElement.style.display = 'block';
        }

        function hideError(field, errorElement) {
            field.classList.remove('invalid');
            field.removeAttribute('aria-invalid');
            field.removeAttribute('aria-describedby');
            errorElement.style.display = 'none';
        }
    </script>

</body>
</html>
```

OBSERVATIONS:



Strengths of the Code

- **Excellent Accessibility (A11y):** This is a standout feature of your code.
  - **Labels:** Every form input is correctly associated with a `<label>` using the `for` and `id` attributes. This is fundamental for screen reader users.
  - **ARIA Roles:** You've correctly used `aria-required="true"` to programmatically signify mandatory fields. The use of `aria-live="polite"` on the error message containers is a superb touch, as it ensures that screen readers will announce validation errors as they appear without being disruptive.
  - **Focus Management:** The custom `:focus` style provides clear visual feedback for keyboard navigators, which is a great accessibility and usability win.
- **Robust JavaScript Validation:**
  - The validation logic is clean, well-organized into separate functions (`validateField`, `validateEmail`, etc.), and easy to read.
  - Using `event.preventDefault()` and `novalidate` on the form tag is the correct way to implement custom validation.
  - The `showError` and `hideError` functions are very well-implemented. They don't just show a message; they also toggle CSS classes and update ARIA attributes (`aria-invalid`, `aria-describedby`), which is the proper way to handle validation state.
- **Modern & Maintainable CSS:**
  - The use of CSS Custom Properties (`:root`) for the color palette and other design tokens is a best practice. It makes the form incredibly easy to re-theme or adjust.
  - The layout, built with Flexbox to center the form, is modern and efficient. The responsive approach (`max-width` and `width: 100%`) is simple and effe
  - The styling is clean, professional, and provides good visual feedback with hover and focus states

Suggestions for Improvement

The code is already excellent, but here are a couple of minor suggestions for taking it from great to perfect.

1. Enhance the User Experience on Success

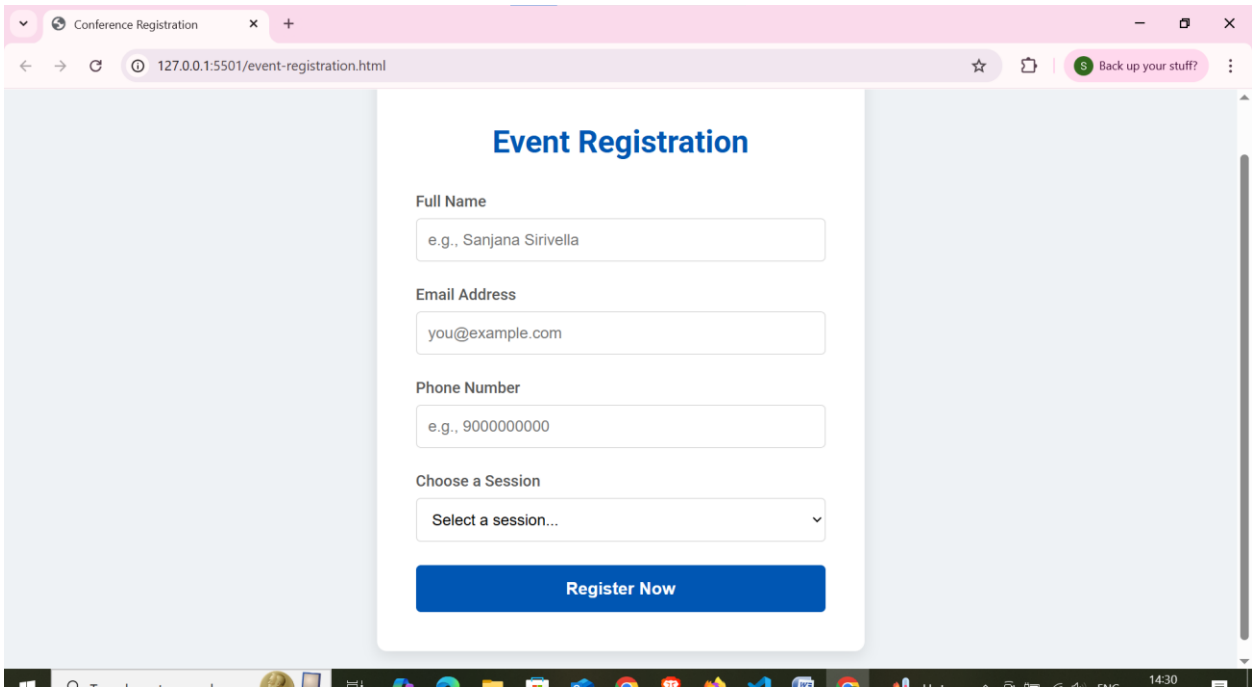
Currently, a browser `alert()` is used for the success message. While functional, alerts can be jarring and don't match the polished feel of the rest of the form. A more elegant solution would be to replace the form with a success message directly on the page.

2. Improve the Phone Number Input Type

Using `type="tel"` is correct, but you can enhance the mobile experience by adding an `inputmode` attribute. Setting `inputmode="numeric"` will hint to mobile browsers that they should display a numeric keypad, which is more convenient for entering phone numbers.

I've prepared a small diff below to show how you could implement these two minor enhancements.

OUTPUT:



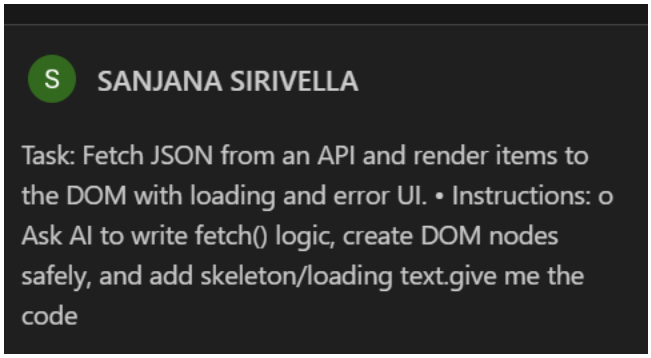
TASK-4: (Data – Fetch API & Render List with Loading/Error States)

• Task: Fetch JSON from an API and render items to the DOM with loading and error UI.

• Instructions:

○ Ask AI to write fetch() logic, create DOM nodes safely, and add skeleton/loading text.

PROMPT:



CODE:

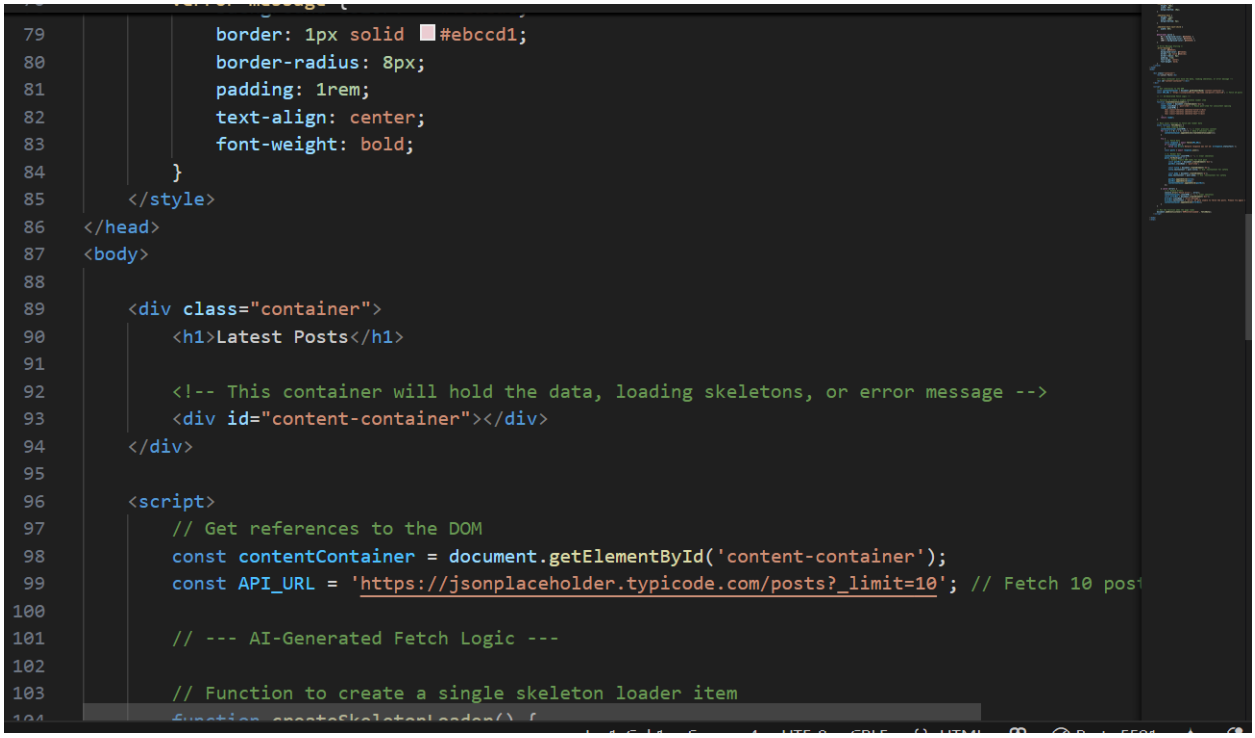
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>API Data Fetching</title>
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Aria
            background-color: #f0f2f5;
            color: #1c1e21;
            margin: 0;
            padding: 2rem;
        }

        .container {
            max-width: 800px;
            margin: 0 auto;
        }

        h1 {
            color: #0056b3;
        }

        /* Styles for each rendered post item */
        .post-item {
            background-color: #fff;
            border: 1px solid #dddfe2;
            border-radius: 8px;
            padding: 1rem;
```

```
.post-item {
    padding: 1rem;
    margin-bottom: 1rem;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.post-item h2 {
    font-size: 1.25rem;
    margin: 0 0 0.5rem 0;
    color: #050505;
}

.post-item p {
    margin: 0;
    line-height: 1.5;
}

/* AI-Suggested Skeleton Loader */
.skeleton {
    background-color: #e0e0e0;
    border-radius: 4px;
    animation: pulse 1.5s infinite ease-in-out;
}

.skeleton-title {
    height: 20px;
    width: 70%;
```

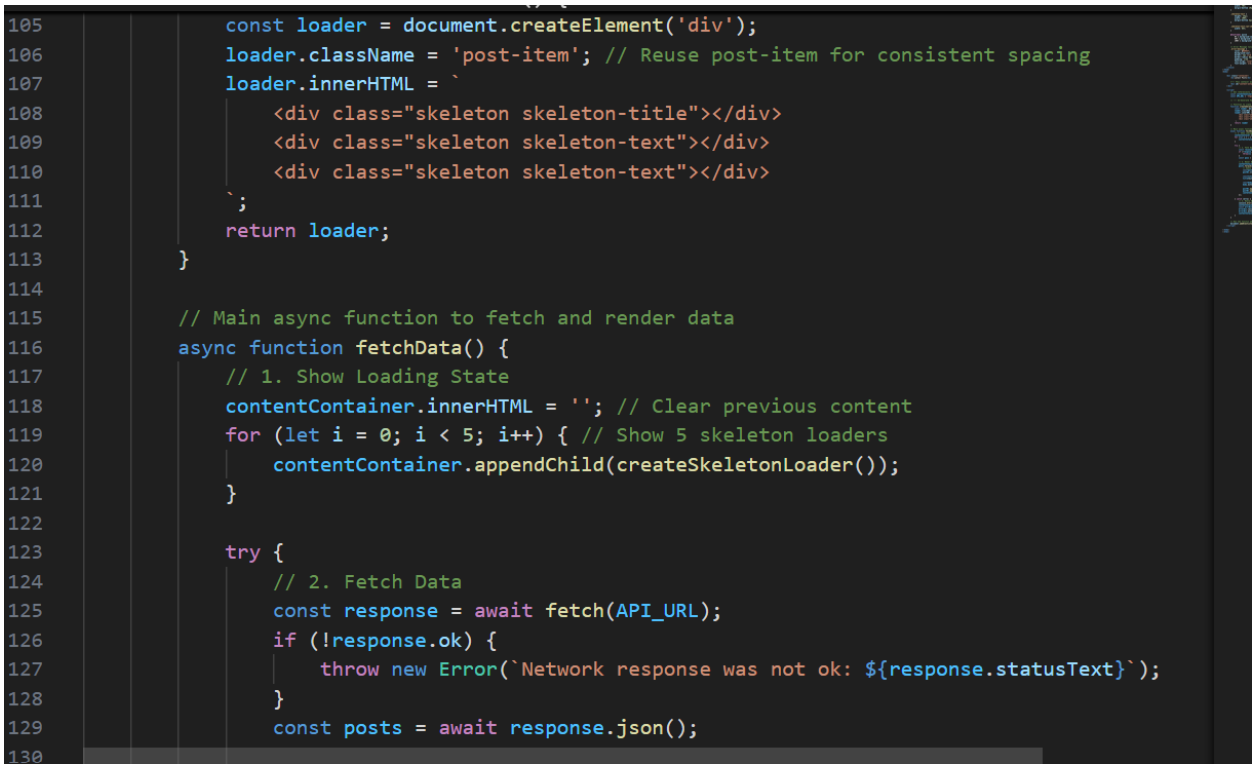
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF { } HTML Port : 5501

```
.skeleton-title {
    height: 20px;
    width: 70%;
    margin-bottom: 10px;
}

.skeleton-text {
    height: 15px;
    width: 100%;
    margin-bottom: 5px;
}

.skeleton-text:last-child {
    width: 85%;
}

@keyframes pulse {
    0% { background-color: #e0e0e0; }
    50% { background-color: #f0f0f0; }
    100% { background-color: #e0e0e0; }
}

/* Error Message Styling */
.error-message {
    color: #d9534f;
    background-color: #f2dede;
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF { } HTML Port : 5501



```
            border: 1px solid #ebccd1;
            border-radius: 8px;
            padding: 1rem;
            text-align: center;
            font-weight: bold;
        }
    </style>
</head>
<body>

    <div class="container">
        <h1>Latest Posts</h1>

        <!-- This container will hold the data, loading skeletons, or error message -->
        <div id="content-container"></div>
    </div>

    <script>
        // Get references to the DOM
        const contentContainer = document.getElementById('content-container');
        const API_URL = 'https://jsonplaceholder.typicode.com/posts?_limit=10'; // Fetch 10 post

        // --- AI-Generated Fetch Logic ---

        // Function to create a single skeleton loader item
```



```
            const loader = document.createElement('div');
            loader.className = 'post-item'; // Reuse post-item for consistent spacing
            loader.innerHTML = `
                <div class="skeleton skeleton-title"></div>
                <div class="skeleton skeleton-text"></div>
                <div class="skeleton skeleton-text"></div>
            `;
            return loader;
        }

        // Main async function to fetch and render data
        async function fetchData() {
            // 1. Show Loading State
            contentContainer.innerHTML = ''; // Clear previous content
            for (let i = 0; i < 5; i++) { // Show 5 skeleton loaders
                contentContainer.appendChild(createSkeletonLoader());
            }

            try {
                // 2. Fetch Data
                const response = await fetch(API_URL);
                if (!response.ok) {
                    throw new Error(`Network response was not ok: ${response.statusText}`);
                }
                const posts = await response.json();
```

```
116         async function fetchData() {
131             // 3. Render Data
132             contentContainer.innerHTML = ''; // Clear skeletons
133             posts.forEach(post => {
134                 // Safely create DOM nodes for each post
135                 const postDiv = document.createElement('div');
136                 postDiv.className = 'post-item';
137
138                 const title = document.createElement('h2');
139                 title.textContent = post.title; // Use .textContent for safety
140
141                 const body = document.createElement('p');
142                 body.textContent = post.body; // Use .textContent for safety
143
144                 postDiv.appendChild(title);
145                 postDiv.appendChild(body);
146                 contentContainer.appendChild(postDiv);
147             });
148
149         } catch (error) {
150             // 4. Handle Errors
151             console.error('Fetch error:', error);
152             contentContainer.innerHTML = ''; // Clear skeletons
153             const errorDiv = document.createElement('div');
154             errorDiv.className = 'error-message';
155             errorDiv.textContent = 'Sorry, we were unable to fetch the posts. Please try ag
156             contentContainer.appendChild(errorDiv);
```

```
148
149         } catch (error) {
150             // 4. Handle Errors
151             console.error('Fetch error:', error);
152             contentContainer.innerHTML = ''; // Clear skeletons
153             const errorDiv = document.createElement('div');
154             errorDiv.className = 'error-message';
155             errorDiv.textContent = 'Sorry, we were unable to fetch the posts. Please try ag
156             contentContainer.appendChild(errorDiv);
157         }
158     }
159
160     // Run the function when the page loads
161     document.addEventListener('DOMContentLoaded', fetchData);
162 </script>
163
164 </body>
165 </html>
```

OBSERVATIONS:

Strengths of the Code

- **Excellent State Management:** The code does a fantastic job of managing the UI for three distinct states:
    1. **Loading State:** It immediately provides visual feedback to the user with skeleton loaders, which is a superior user experience compared to a blank screen or a simple "Loading..." message.
    2. **Success State:** It safely creates and appends new DOM elements to display the fetched data.
    3. **Error State:** It gracefully handles potential network or parsing errors within a `try...catch` block and displays a clear, user-friendly error message.
- **Security-First DOM Manipulation:** You've used `document.createElement()` and `element.textContent` to render the data. This is a critical security best practice. By using `.textContent`, you ensure that any text from the API is treated as plain text, which automatically prevents Cross-Site Scripting (XSS) attacks that could occur if you used `.innerHTML` with unsanitized data.
- **Modern JavaScript (`async/await`):** The use of `async/await` makes the asynchronous logic clean, readable, and easy to follow, avoiding the "pyramid of doom" that can sometimes happen with older callback or promise-chaining patterns.
- **Clean CSS and Animations:** The skeleton loader is simple but very effective. The `pulse` keyframe animation provides a subtle visual cue that something is happening in the background, and the styling is clean and modern.

GEMINI CODE ASSIST: CHAT

Suggestions for Improvement

The code is already very strong, but here are a couple of minor refinements to enhance accessibility and code structure.

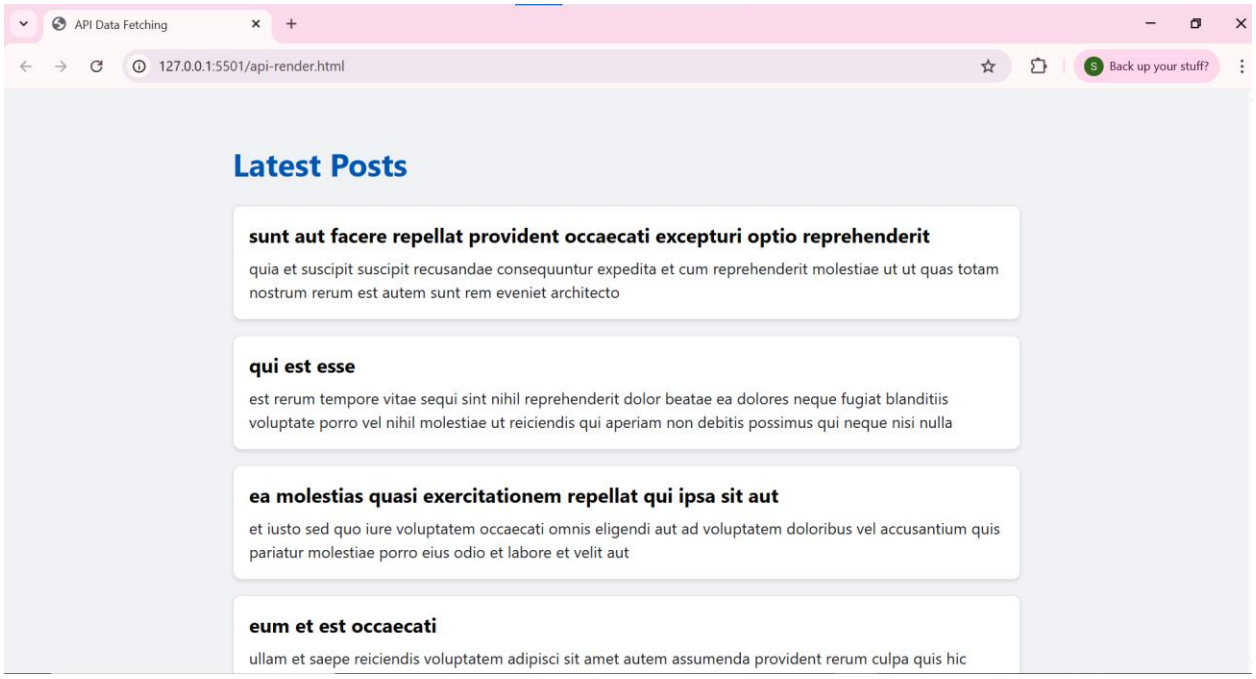
1. Enhance Accessibility for Screen Readers

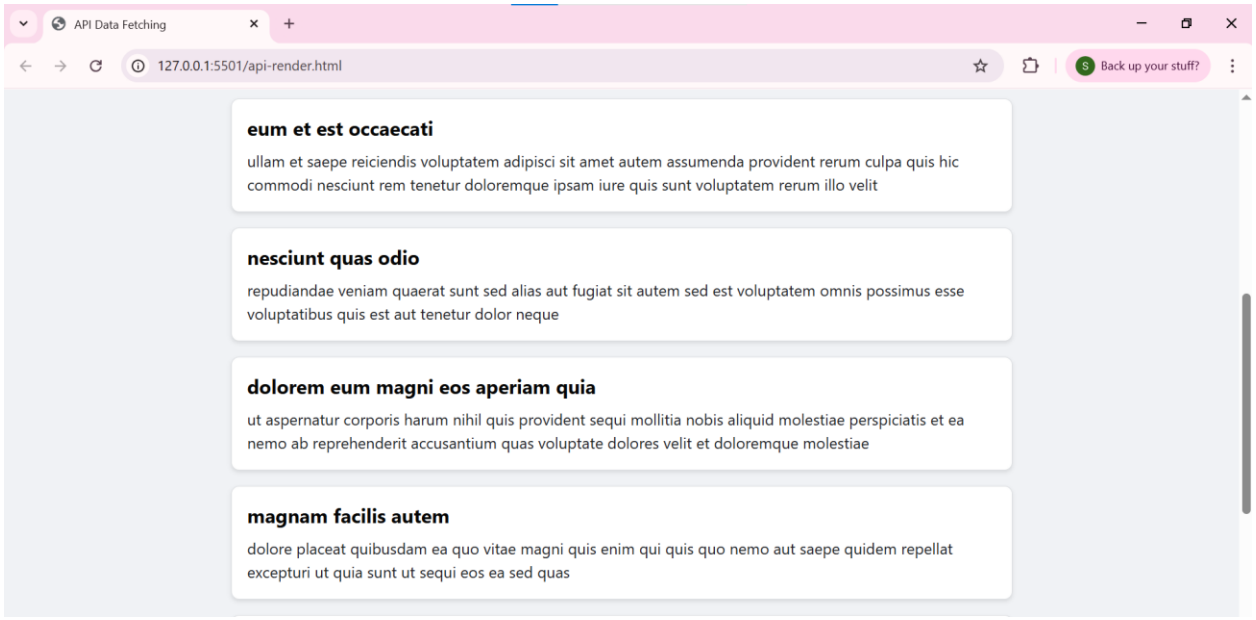
While the skeleton loaders are great visually, they don't communicate the "loading" state to users of assistive technologies like screen readers. We can easily fix this by adding ARIA attributes to the container. By setting `aria-busy="true"` during the fetch and `aria-busy="false"` when it's complete, we can make the state programmatically available.

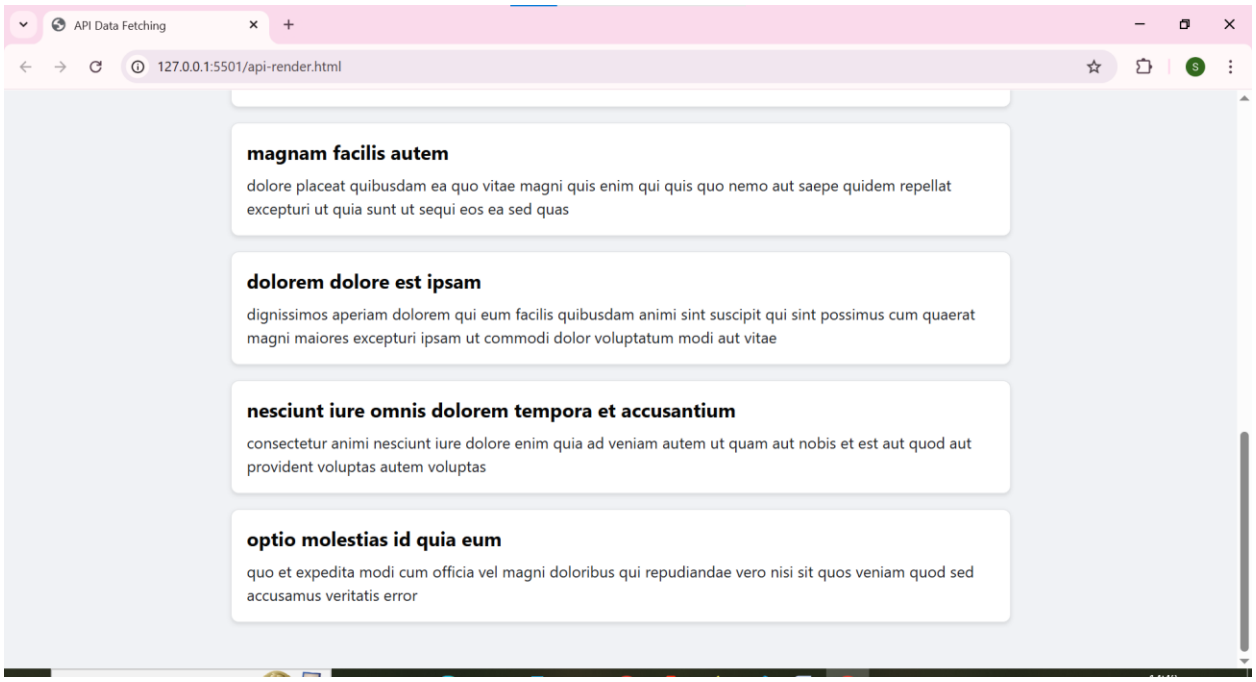
2. Consolidate DOM Clearing

The line `contentContainer.innerHTML = '';` is called in both the `try` and `catch` blocks before rendering new content. While this works perfectly, you can slightly streamline the logic by clearing the container just once before the `try...catch` block begins.

OUTPUT:




API Data Fetching
127.0.0.1:5501/api-render.html
Back up your stuff?
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit
nesciunt quas odio
repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque
dolorem eum magni eos aperiam quia
ut aspernatur corporis harum nihil quis provident sequi mollitia nobis aliquid molestiae perspiciatis et ea nemo ab reprehenderit accusantium quas voluptate dolores velit et doloremque molestiae
magnam facilis autem
dolore placeat quibusdam ea quo vitae magni quis enim qui quis quo nemo aut saepe quidem repellat excepturi ut quia sunt ut sequi eos ea sed quas


API Data Fetching
127.0.0.1:5501/api-render.html
magnam facilis autem
dolore placeat quibusdam ea quo vitae magni quis enim qui quis quo nemo aut saepe quidem repellat excepturi ut quia sunt ut sequi eos ea sed quas
dolorem dolore est ipsam
dignissimos aperiam dolorem qui eum facilis quibusdam animi sint suscipit qui sint possimus cum quaerat magni maiores excepturi ipsam ut commodi dolor voluptatum modi aut vitae
nesciunt iure omnis dolorem tempora et accusantium
consectetur animi nesciunt iure dolore enim quia ad veniam autem ut quam aut nobis et est aut quod aut provident voluptas autem voluptas
optio molestias id quia eum
quo et expedita modi cum officia vel magni doloribus qui repudiandae vero nisi sit quos veniam quod sed accusamus veritatis error

----------THE END----------